

Fast compositional graph parsing with AM dependency trees

Matthias Lindemann, Jonas Groschwitz, Alexander Koller

Department of Language Science and Technology
Saarland University



AM Dependency Parsing (Groschwitz et al. 2018)

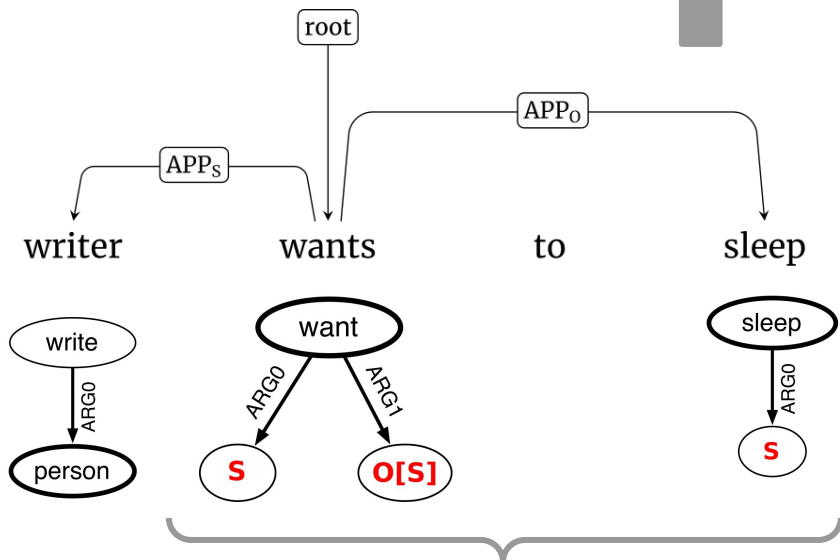
- Compositional method for graph parsing.
- Applied to AMR, DM, PAS, PSD, EDS and UCCA with high accuracies (Lindemann et al. 2019, Donatelli et al. 2019).

In this talk:

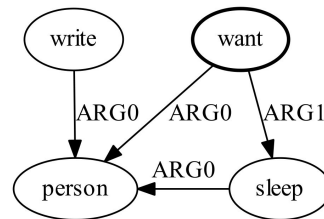
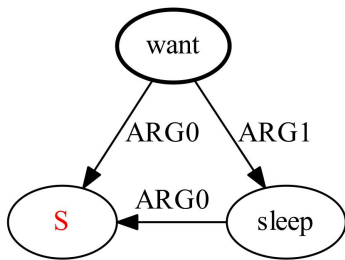
Fast methods for AM dependency parsing.

AM Dependency Trees

The



Root: **bold**
Sources: **red**



Operations

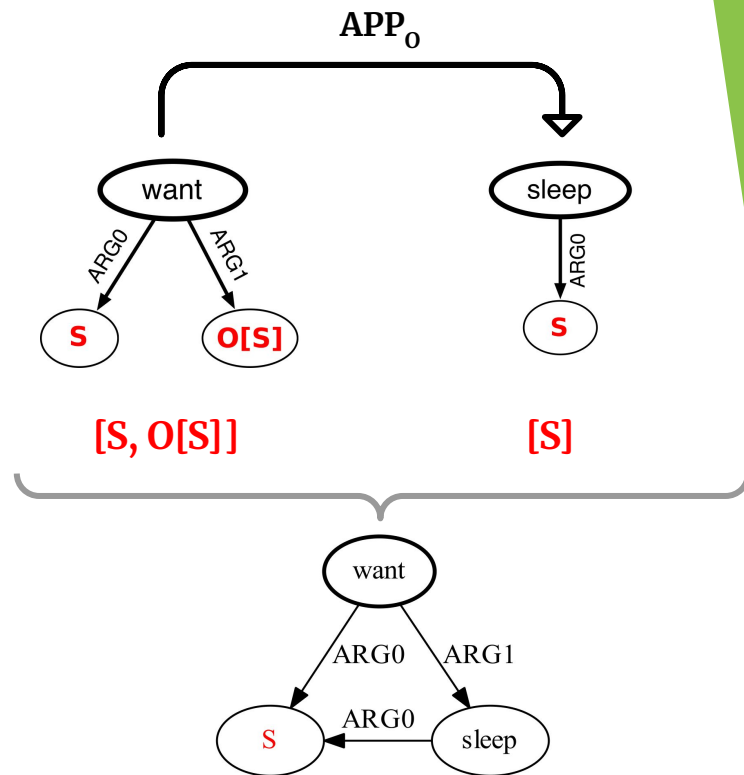
Graph constants

AM Algebra: Types

Type of a graph = set of its sources + annotations

Types...

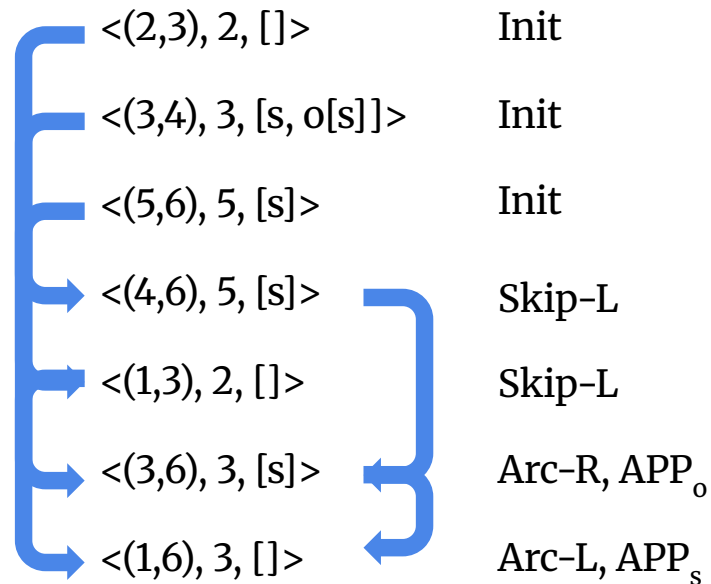
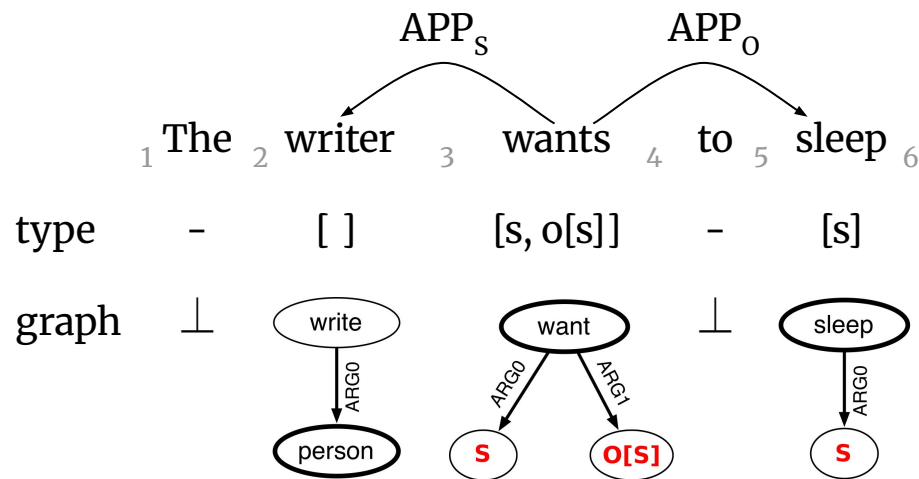
- ▶ ... restrict which arguments are allowed.
- ▶ ... control reentrancies.
- ▶ ... effectively guide the parser.



AM Dependency Parsing

1. Obtain latent AM dependency trees for training corpus (Lindemann et al. 2019).
 2. Train parser to predict AM dependency trees (Groschwitz et al. 2018).
 3. Evaluation on unseen sentence:
 - ▷ Find best *well-typed* AM dependency tree.
 - ▷ Evaluate (deterministically) to graph.
- so far: slow!
- Type system makes parsing NP-complete.
 - We have two approximate unpruned chart parsers.
 - Feasible but still slow runtimes: More than 4½ h to parse AMR test set – and skipped two sentences.

Projective parser



A* parser

- ▶ Speed up parser by sorting agenda wrt item cost + estimated cost to complete item (Klein & Manning 03; Lewis et al. 16).
- ▶ We use neural supertag- and edge-factored cost model:

$$\text{cost}(t) = \sum_k \text{cost}(\text{graph at } k) + \text{cost}(\text{edge into } k)$$

- ▶ Heuristics for estimating outside costs:
 - ▷ sum costs of best supertag for each token outside the item.
 - ▷ + cost of best edge into each token outside the item.
 - ▷ certain combinations of edges and supertags are not allowed.

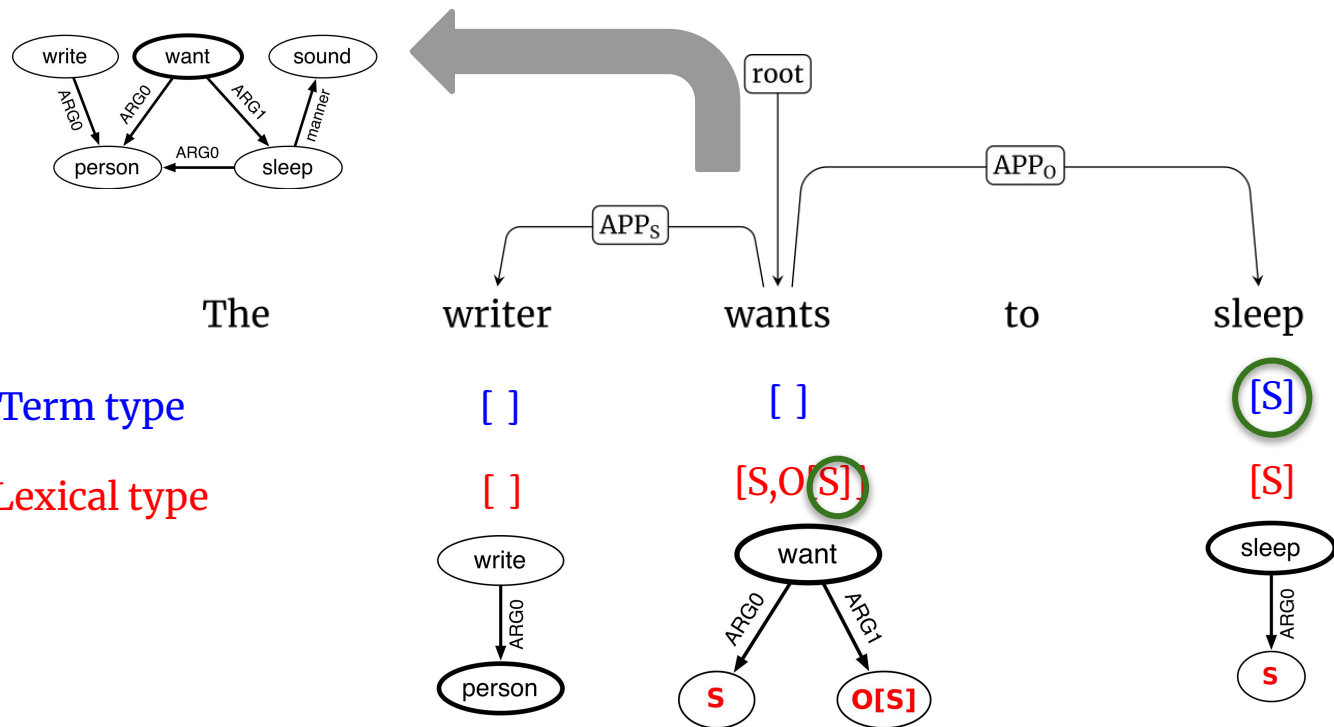
Transition-based parsing

- ▶ Fast alternative to chart parsing.
- ▶ We parse top-down:
 - ▷ draw all outgoing edges of node.
 - ▷ determine its graph constant.
 - ▷ recursively continue with all children.
- ▶ No projectivity constraints.
- ▶ Prob. model inspired by neural top-down parser of Ma et al. (2018).
- ▶ Run-time is $O(n^2)$.

Lexical types and term types

Lexical type: type of the graph constant.

Term type: type of graph the subtree evaluates to.



Source annotations must match with term types of corresponding children!

Parsing example

Stack:

sleep
writer

The writer wants to sleep

Term types

[]

[]

[S]

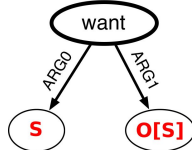
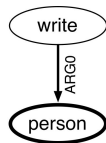
Consistent
lexical types

{[], [S],
[S, []]}

{[], [S],
[S, O[S]]}

{[], [S],
[S, O[S]]}

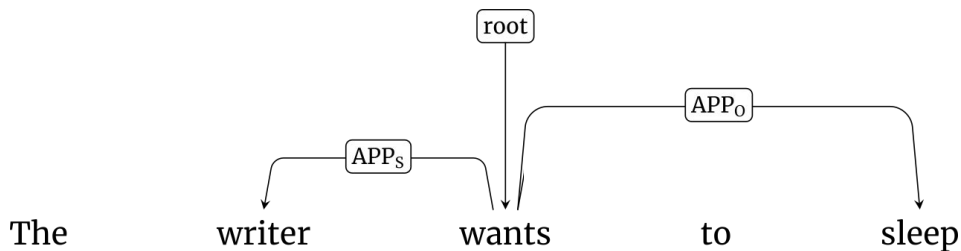
Graph constants



Dead ends I

Stack:

wants	
-------	--



Term types

[]

Consistent
lexical types

{~~H~~, {S},
{S, O{S}}}

Graph constants

Problem:

No consistent lexical type, cannot
choose graph constant!

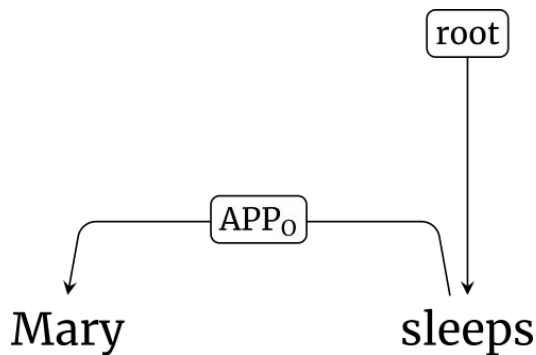
Solution:

Only allow operations such that
Consistent lexical types *never*
becomes empty.

Dead ends II

Stack:

sleeps	
--------	--



Term types

`[]`

Consistent
lexical types

~~{`H`, `S`}~~,
`[S,O[S]]`}

Graph constants

Problem:

- Only consistent type is `[S,O[S]]`
- We have to add the APP_s child!
- but: no tokens left.

Solution:

Only allow operations such that:

min. # APP needed to complete $\leq$ # tokens left

Theoretical guarantees

1. Every derived AM dependency tree is well-typed
2. All well-typed AM dependency trees can be derived
3. No dead ends: Every parser configuration can be completed

Assumptions needed:

- no hard restrictions which graph constant to assign to a token.
- there is a graph constant for every term type that may arise.

In practice: need to create up to 14 additional graph constants, increases graph lexicon by < 1%.

Evaluation

- AM dependency trees and costs for A* come from parser of Lindemann et al. (2019).
- Evaluate two transition systems:
 - Lexical Type Last (LTL)
 - Lexical Type First (LTF) (not presented here)

Parsing accuracy

		DM		PAS		PSD		EDS		AMR 15	AMR 17
		id F	ood F	id F	ood F	id F	ood F	Smatch F	EDM	Smatch F	Smatch F
Non-proj.	He and Choi (2020) [♣]	94.6	90.8	96.1	94.4	86.8	79.5	-	-	-	-
	Chen et al. (2018)	-	-	-	-	-	-	90.9	90.4	-	-
	Cai and Lam (2020) [♣]	-	-	-	-	-	-	-	-	-	80.2
	Zhang et al. (2019) [♣]	92.2	87.1	-	-	-	-	-	-	-	77.0±0.1
	L'19 [♣] , w/o MTL	93.9±0.1	90.3±0.1	94.5±0.1	92.5±0.1	82.0±0.1	81.5±0.3	90.1±0.1	84.9±0.1	75.1±0.1	76.0±0.2
Projective	A* parser [♣]	91.6±0.1	88.2±0.2	94.4±0.1	92.6±0.1	81.6±0.1	81.5±0.2	87.5±0.6	82.8±0.1	74.2±0.1	75.1±0.1
	LTL [♣] , no types	88.1±0.3	82.5±0.4	88.4±0.4	83.3±0.9	66.4±0.7	66.0±0.5	80.9±0.6	76.7±0.5	38.6±0.8	45.7±1.4
	LTL [♣] , greedy	93.8±0.1	90.1±0.1	94.6±0.1	92.5±0.2	81.4±0.1	80.0±0.4	90.3±0.2	84.9±0.1	74.8±0.1	76.0±0.1
	beam=3	94.0±0.1	90.4±0.1	94.7±0.1	92.8±0.2	81.9±0.1	81.4±0.2	90.4±0.1	85.1±0.1	75.7±0.2	76.9±0.1
	LTF [♣] , greedy	92.1±0.1	88.0±0.1	93.8±0.2	91.0±0.4	76.8±0.2	75.4±0.5	87.9±0.1	82.9±0.1	68.6±0.4	70.7±0.4
	beam = 3	93.8±0.2	90.4±0.2	94.6±0.2	92.6±0.2	81.3±0.1	81.0±0.1	90.2±0.3	85.0±0.2	74.7±0.2	75.8±0.4

Table 1: Semantic parsing accuracies (id = in domain test set; ood = out of domain test set). [♣] marks models using BERT. L'19 are results reported by [Lindemann et al. \(2019\)](#) with fixed tree decoder (incl. post-processing bugfix).

Run times

	DM	PAS	PSD	EDS	AMR 15	AMR 17
projective, L'19♠costs	3	2	4	4	<2	<2
L'19♠fixed tree	710	97	265	542	<4	<3
A*♠, ignore-aware	712	2167	1318	136	<22	<30
A*♠, edge-based	725	2105	1421	129	<20	<26
A*♠, trivial	706	2096	1235	105	<9	<10
LTL♠, greedy	1020	844	1028	949	934	891
beam=3	176	155	177	171	210	199
LTL♠, greedy, GPU	4539	4321	4158	4503	2346	2721
LTF♠, greedy	840	799	665	691	561	518
beam=3	124	110	88	99	102	91

Table 5.1: Avg. parsing speed in tokens/s on test sets. < indicates where parsing was interrupted due to timeout.

Conclusion

- Two methods for AM dependency parsing:
 - Projective A* parsing.
 - Top-down transition-based parsing.
 - No dead ends.
- Both methods guarantee well-typedness.
- High accuracies, on-par or better than our previous parser.
- Reduce parsing time from multiple hours to about 10 seconds.